

PROJECT DOCUMENTATION

Data Warehouse & Analytics Project

Building an Enterprise-Grade SQL Server Data Warehouse using the Medallion Architecture

Prepared By	Elvis Titus Cherukara
Project	Data Warehouse and Analytics using SQL Server
Technology Stack	SQL Server, SSMS, T-SQL
Architecture	Medallion Architecture (Bronze → Silver → Gold)

Table of Contents

1. Introduction

- 1.1 Project Motivation & Selection Rationale
- 1.2 Core Objectives

2. Data Architecture Overview

3. Source Systems

- 3.1 CRM Source System
- 3.2 ERP Source System

4. Bronze Layer — Raw Data Ingestion

- 4.1 Overview & Design Principles
- 4.2 Source Schema Specifications
- 4.3 Data Flow Architecture
- 4.4 Implementation Scripts
- 4.5 Execution & Operational Logging

5. Silver Layer — Cleansing & Standardization

- 5.1 Overview & Objectives
- 5.2 Source-to-Target Mapping Rules
- 5.3 Implementation Scripts
- 5.4 Execution & Logging

6. Gold Layer — Star Schema

- 6.1 Overview & Objectives
- 6.2 Star Schema Diagram
- 6.3 Dimension & Fact Logic Specifications
- 6.4 Implementation Scripts
- 6.5 Data Quality & Testing Checks

7. Summary

1. Introduction

1.1 Project Motivation & Selection Rationale

Why Choose This Project?

Most academic or beginner SQL projects focus solely on writing queries for existing, clean datasets — for example, SELECT queries for exploratory analysis. In enterprise settings, however, raw data is rarely clean, well-structured, or centralized.

This project was selected to **bridge the gap between theoretical SQL querying and real-world Data Engineering**, demonstrating how to design and build an enterprise-grade Data Warehouse from scratch.

- **End-to-End Implementation:** Covers the full lifecycle of data — from extracting messy CSV sources to designing a multidimensional data model tailored for business intelligence.
- **Industry-Standard Architecture:** Implements the modern Medallion Architecture (Bronze → Silver → Gold) used by leading enterprise platforms such as Databricks and Snowflake.
- **Multi-Role Mastery:** Demonstrates practical expertise across three distinct data roles:
 - **Data Architect** — designing data pipelines, layer abstractions, and enforcement of Separation of Concerns.
 - **Data Engineer** — writing robust SQL ETL scripts, handling string manipulations, parsing dates, standardizing invalid inputs, and logging load runtimes.
 - **Data Modeler** — developing optimized Star Schemas using Dimension and Fact tables.
- **Solving Real Business Chaos:** Directly addresses common organizational failures — isolated Excel/PowerPoint reports, mismatched data metrics, slow manual reporting, and lack of historical tracking.

1.2 Core Objectives

The primary objective of this project is to transform fragmented, low-quality transactional data into a centralized, highly reliable, and optimized Data Warehouse.

Key Goals & Deliverables

- **Centralize Disparate Systems:** Seamlessly ingest and integrate data from multiple decoupled source systems (CRM and ERP) into a single point of truth.
- **Standardize & Cleanse Data:** Resolve data quality issues across sources — missing values, inconsistent dates, unmapped codes, and duplicate records.
- **Build a Scalable Pipeline:** Create idempotent, reusable, and automated SQL Stored Procedures for ETL/ELT processing with error-handling mechanisms.
- **Optimize for Analytics & BI:** Transform flat, normalized source structures into a Star Schema to drastically improve analytical query performance and simplify report creation.

- **Ensure Data Quality & Governance:** Implement strict Quality Control (QC) testing scripts to validate primary/foreign key relationships, record counts, and surrogate key uniqueness.

2. Data Architecture Overview

The project follows the architecture shown below, moving data progressively through three layers of increasing quality and business value.

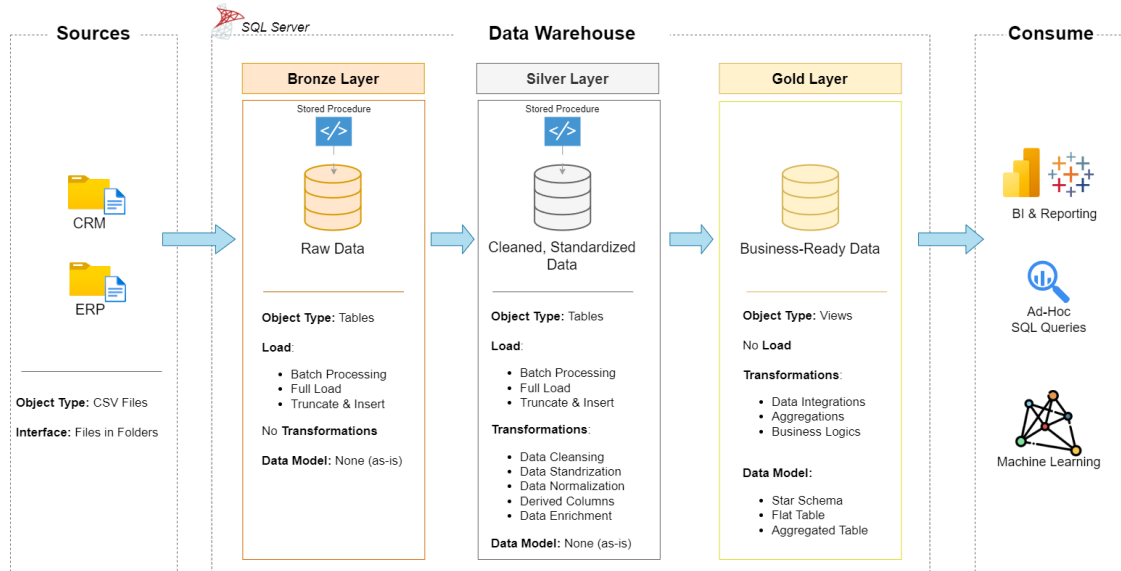


Figure 2.1 — Data Warehouse Architecture using the Medallion Architecture

Layer Descriptions

Bronze Layer (Raw Landing)

- Ingests raw CSV source data as-is, without changing formats or types.
- Uses NVARCHAR types to prevent ingestion failures and guarantees full execution traceability via TRUNCATE AND BULK INSERT operations inside stored procedures.

Silver Layer (Cleansing & Transformation)

- Cleans bad data, handles missing attributes, parses strings to correct data types (DATE, INT, DECIMAL), trims whitespace, and normalizes categorical values.
- Combines CRM and ERP customer/product records into single, unified entities.

Gold Layer (Business Analytics)

- Models cleansed data into a Star Schema consisting of a central Fact Table (gold.fact_sales) surrounded by Dimension Tables (gold.dim_customers, gold.dim_products).
- Utilizes Surrogate Keys for efficient joins and protects reporting queries from source schema changes.

3. Source Systems

The first step of the project was to collect the source datasets required for building the Data Warehouse. The project draws from two independent business systems.

3.1 CRM Source System

The CRM system contains customer, product, and sales information.

Files Used

File	Description
cust_info.csv	Customer information
prd_info.csv	Product information
sales_details.csv	Sales transactions

Activities Performed

- Collected CRM datasets.
- Verified CSV format.
- Reviewed file structure.
- Organized files inside the project directory.

3.2 ERP Source System

The ERP system provides additional business information that complements the CRM data.

Files Used

File	Description
CUST_AZ12.csv	Customer demographic details
LOC_A101.csv	Customer location details
PX_CAT_G1V2.csv	Product category details

Activities Performed

- Collected ERP datasets.
- Verified CSV format.
- Examined data structure.
- Stored files in the project directory.

4. Bronze Layer — Raw Data Ingestion

4.1 Overview & Design Principles

The Bronze Layer acts as the initial landing zone (staging area) for raw data entering the Data Warehouse from external source systems (CRM and ERP CSV files).

Key Design Principles

- **Raw Replication ("As-Is"):** Data is loaded into SQL Server without altering values, data types, or structure.
- **No Quality Enforcement:** Cleansing, deduplication, and data transformation are explicitly postponed to the Silver layer.
- **Full Refresh Strategy (Truncate & Load):** Tables are completely truncated prior to loading to guarantee idempotent re-executions.
- **Encapsulation via Stored Procedures:** The entire ingestion process is managed and monitored programmatically using SQL Stored Procedures.

4.2 Source Schema Specifications

The ingestion architecture draws from two primary source domains, split across multiple CSV flat files.

Source System A: CRM (Customer Relationship Management)

Bronze Table	Source File	Description	Target Column Types
bronze.crm_cust_info	cust_info.csv	Customer demographics, creation dates, & status	NVARCHAR(50) for all fields
bronze.crm_prd_info	prd_info.csv	Product details, categories, & pricing info	NVARCHAR(50) for all fields
bronze.crm_sales_details	sales_details.csv	Customer transactions & order history	NVARCHAR(50) for all fields

Source System B: ERP (Enterprise Resource Planning)

Bronze Table	Source File	Description	Target Column Types
bronze.erp_cust_az12	CUST_AZ12.csv	ERP customer cross-reference & birthdates	NVARCHAR(50) for all fields
bronze.erp_loc_a101	LOC_A101.csv	Geographic/location mappings per customer	NVARCHAR(50) for all fields
bronze.erp_px_cat_g1v2	PX_CAT_G1V2.csv	Product category structure and maintenance attributes	NVARCHAR(50) for all fields

Note: Text types such as NVARCHAR(50) / VARCHAR(50) are used intentionally in the Bronze DDL to avoid runtime load failures caused by formatting errors or mismatched source data types.

4.3 Data Flow Architecture

The diagram below traces data lineage across all three layers — from the CRM and ERP source files, through the Bronze and Silver tables, into the final Gold layer dimension and fact objects.

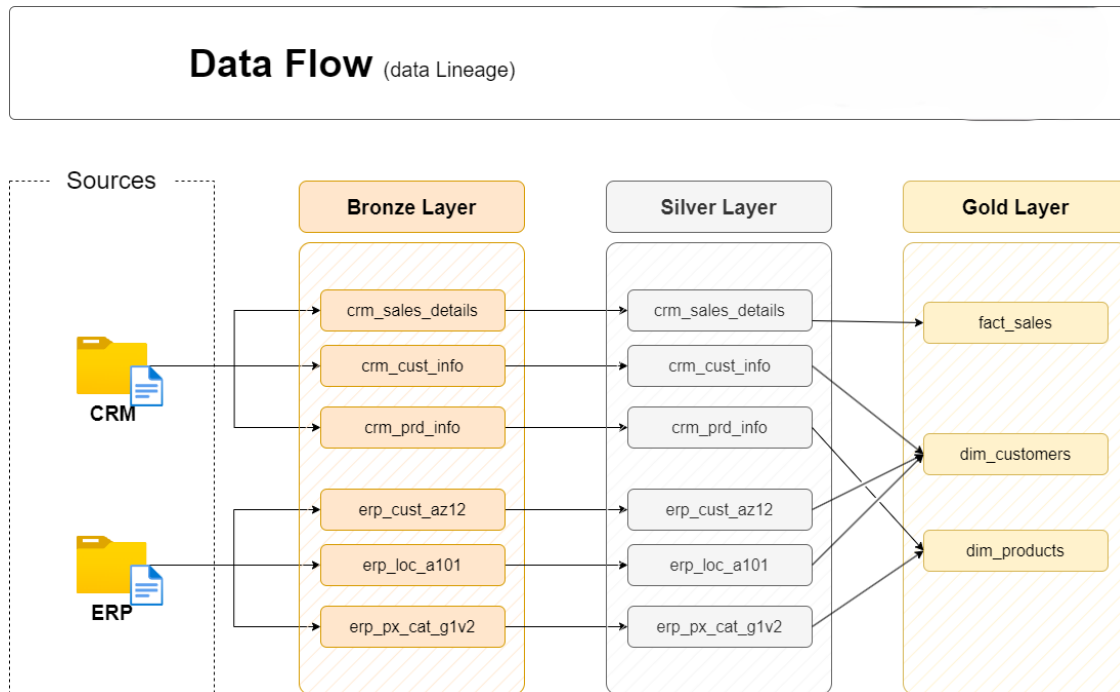


Figure 4.1 — Data Flow / Data Lineage across Bronze, Silver, and Gold layers

4.4 Implementation Scripts

Step 4.4.1 — Database & Schema Initialization (init_database.sql)

```
SQL
CREATE DATABASE DataWarehouse;
GO
USE DataWarehouse;
GO

-- Create Schemas for Medallion Architecture
CREATE SCHEMA bronze;
GO
CREATE SCHEMA silver;
GO
CREATE SCHEMA gold;
GO
```

Step 4.4.2 — Bronze DDL Definition (ddl_bronze.sql)

```
SQL
USE DataWarehouse;
GO

IF OBJECT_ID('bronze.crm_cust_info', 'U') IS NOT NULL DROP TABLE bronze.crm_cust_info;
CREATE TABLE bronze.crm_cust_info (
    cst_id NVARCHAR(50),
```



```

        cst_key            NVARCHAR(50),
        cst_firstname      NVARCHAR(50),
        cst_lastname       NVARCHAR(50),
        cst_marital_status NVARCHAR(50),
        cst_gndr           NVARCHAR(50),
        cst_create_date    NVARCHAR(50)
    );

IF OBJECT_ID('bronze.crm_prd_info', 'U') IS NOT NULL DROP TABLE bronze.crm_prd_info;
CREATE TABLE bronze.crm_prd_info (
    prd_id      NVARCHAR(50),
    prd_key     NVARCHAR(50),
    prd_nm      NVARCHAR(50),
    prd_cost    NVARCHAR(50),
    prd_line    NVARCHAR(50),
    prd_start_dt NVARCHAR(50),
    prd_end_dt  NVARCHAR(50)
);

IF OBJECT_ID('bronze.crm_sales_details', 'U') IS NOT NULL DROP TABLE bronze.crm_sales_details;
CREATE TABLE bronze.crm_sales_details (
    sls_ord_num NVARCHAR(50),
    sls_prd_key NVARCHAR(50),
    sls_cust_id NVARCHAR(50),
    sls_order_dt NVARCHAR(50),
    sls_ship_dt  NVARCHAR(50),
    sls_due_dt   NVARCHAR(50),
    sls_sales    NVARCHAR(50),
    sls_quantity NVARCHAR(50),
    sls_price    NVARCHAR(50)
);

IF OBJECT_ID('bronze.erp_cust_az12', 'U') IS NOT NULL DROP TABLE bronze.erp_cust_az12;
CREATE TABLE bronze.erp_cust_az12 (
    cid  NVARCHAR(50),
    bdate NVARCHAR(50),
    gen  NVARCHAR(50)
);

IF OBJECT_ID('bronze.erp_loc_a101', 'U') IS NOT NULL DROP TABLE bronze.erp_loc_a101;
CREATE TABLE bronze.erp_loc_a101 (
    cid  NVARCHAR(50),
    cntry NVARCHAR(50)
);

IF OBJECT_ID('bronze.erp_px_cat_g1v2', 'U') IS NOT NULL DROP TABLE bronze.erp_px_cat_g1v2;
CREATE TABLE bronze.erp_px_cat_g1v2 (
    id      NVARCHAR(50),
    cat     NVARCHAR(50),
    subcat  NVARCHAR(50),
    maintenance NVARCHAR(50)
);
GO

```

Step 4.4.3 — Loading Procedure (proc_load_bronze.sql)

SQL

```

CREATE OR ALTER PROCEDURE bronze.load_bronze AS
BEGIN
    DECLARE @start_time DATETIME, @end_time DATETIME,
            @batch_start_time DATETIME, @batch_end_time DATETIME;
    BEGIN TRY

```

```

SET @batch_start_time = GETDATE();
PRINT '=====';
PRINT 'Loading Bronze Layer';
PRINT '=====';

-- CRM System Processing
PRINT '-----';
PRINT 'Loading CRM Tables';
PRINT '-----';

SET @start_time = GETDATE();
PRINT '>> Truncating and Bulk Inserting: bronze.crm_cust_info';
TRUNCATE TABLE bronze.crm_cust_info;
BULK INSERT bronze.crm_cust_info
FROM 'C:\sql-data-warehouse-project\datasets\source_crm\cust_info.csv'
WITH (FIRSTROW = 2, FIELDTERMINATOR = ',', TABLOCK);
SET @end_time = GETDATE();
PRINT '>> Load Duration: ' + CAST(DATEDIFF(second, @start_time, @end_time) AS NVARCHAR) +
' seconds';

-- Additional tables (crm_prd_info, crm_sales_details, erp_*)
-- follow the identical TRUNCATE + BULK INSERT pattern above.

SET @batch_end_time = GETDATE();
PRINT '=====';
PRINT 'Bronze Layer Load Completed Successfully';
PRINT 'Total Load Duration: ' + CAST(DATEDIFF(second, @batch_start_time, @batch_end_time)
AS NVARCHAR) + ' seconds';
PRINT '=====';
END TRY
BEGIN CATCH
PRINT '=====';
PRINT 'ERROR OCCURRED DURING BRONZE LOAD';
PRINT 'Error Message: ' + ERROR_MESSAGE();
PRINT '=====';
END CATCH
END;
GO

```

4.5 Execution & Operational Logging

To run the full raw load process end-to-end, execute the following stored procedure:

```

SQL
EXEC bronze.load_bronze;

```

Sample Output Log

```

Console Output
=====
Loading Bronze Layer
=====
Loading CRM Tables
-----
>> Truncating and Bulk Inserting: bronze.crm_cust_info
>> Load Duration: 1 seconds
>> Truncating and Bulk Inserting: bronze.crm_prd_info
>> Load Duration: 1 seconds

```

```
>> Truncating and Bulk Inserting: bronze.crm_sales_details
>> Load Duration: 3 seconds
```

```
-----
Loading ERP Tables
```

```
-----
>> Truncating and Bulk Inserting: bronze.erp_cust_az12
>> Load Duration: 1 seconds
>> Truncating and Bulk Inserting: bronze.erp_loc_a101
>> Load Duration: 1 seconds
>> Truncating and Bulk Inserting: bronze.erp_px_cat_g1v2
>> Load Duration: 1 seconds
```

```
=====
Bronze Layer Load Completed Successfully
Total Load Duration: 8 seconds
```

5. Silver Layer — Cleansing & Standardization

5.1 Overview & Objectives

The Silver Layer acts as the data transformation and cleansing zone. It takes the raw, unstructured, string-based data from the Bronze Layer, applies business logic, cleans quality defects, handles missing values, standardizes date formats, and aligns codes across the CRM and ERP source systems.

Key Operations Performed

- **Data Cleansing & Trimming:** Removing unwanted leading/trailing whitespace and invalid characters.
- **Type Casting & Standardization:** Explicitly casting all fields from NVARCHAR into native data types (INT, DATE, DECIMAL).
- **Categorical Normalization:** Unifying varied source representations (e.g., mapping M/F, Male/Female into standardized codes like 'Male' or 'Female').
- **Source Integration:** Deduplicating and resolving cross-system keys between CRM and ERP customer/product data.
- **Idempotent Load Processing:** Utilizing TRUNCATE TABLE before executing INSERT INTO ... SELECT statements.

5.2 Source-to-Target Mapping Rules

Bronze Source	Silver Target	Key Transformations & Cleansing Rules
bronze.crm_cust_info	silver.crm_cust_info	Trim spaces on strings. Clean cst_gndr (M→Male, F→Female, else n/a). Clean cst_marital_status (S→Single, M→Married, else n/a). Cast cst_create_date to DATE.
bronze.crm_prd_info	silver.crm_prd_info	Extract cat_id from product key prefix. Cast prd_cost to DECIMAL(13,2). Standardize prd_line codes (M→Mountain, R→Road, etc.). Handle missing/invalid prd_end_dt values.
bronze.crm_sales_details	silver.crm_sales_details	Validate/cast dates (sls_order_dt, sls_ship_dt, sls_due_dt). Recalculate or clean sls_sales, sls_quantity, sls_price (handling negative/null values).
bronze.erp_cust_az12	silver.erp_cust_az12	Strip 'NAS' prefix from cid to align with CRM customer IDs. Clean invalid future birthdates (bdate).
bronze.erp_loc_a101	silver.erp_loc_a101	Strip hyphen '-' from cid.

Bronze Source	Silver Target	Key Transformations & Cleansing Rules
		Standardize country names (e.g., US/USA→United States, DE→Germany).
bronze.erp_px_cat_g1v2	silver.erp_px_cat_g1v2	Direct string trimming and normalization.

5.3 Implementation Scripts

Step 5.3.1 — Silver DDL Definition (ddl_silver.sql)

SQL

```
USE DataWarehouse;
GO

IF OBJECT_ID('silver.crm_cust_info', 'U') IS NOT NULL DROP TABLE silver.crm_cust_info;
CREATE TABLE silver.crm_cust_info (
    cst_id            INT,
    cst_key           NVARCHAR(50),
    cst_firstname     NVARCHAR(50),
    cst_lastname      NVARCHAR(50),
    cst_marital_status NVARCHAR(50),
    cst_gndr          NVARCHAR(50),
    cst_create_date   DATE,
    dwh_create_date   DATETIME DEFAULT GETDATE()
);

IF OBJECT_ID('silver.crm_prd_info', 'U') IS NOT NULL DROP TABLE silver.crm_prd_info;
CREATE TABLE silver.crm_prd_info (
    prd_id           INT,
    cat_id           NVARCHAR(50),
    prd_key          NVARCHAR(50),
    prd_nm           NVARCHAR(50),
    prd_cost         DECIMAL(13,2),
    prd_line         NVARCHAR(50),
    prd_start_dt     DATE,
    prd_end_dt       DATE,
    dwh_create_date  DATETIME DEFAULT GETDATE()
);

IF OBJECT_ID('silver.crm_sales_details', 'U') IS NOT NULL DROP TABLE silver.crm_sales_details;
CREATE TABLE silver.crm_sales_details (
    sls_ord_num      NVARCHAR(50),
    sls_prd_key      NVARCHAR(50),
    sls_cust_id      INT,
    sls_order_dt     DATE,
    sls_ship_dt      DATE,
    sls_due_dt       DATE,
    sls_sales        DECIMAL(13,2),
    sls_quantity     INT,
    sls_price        DECIMAL(13,2),
    dwh_create_date  DATETIME DEFAULT GETDATE()
);

IF OBJECT_ID('silver.erp_cust_az12', 'U') IS NOT NULL DROP TABLE silver.erp_cust_az12;
CREATE TABLE silver.erp_cust_az12 (
    cid             NVARCHAR(50),
    bdate          DATE,
    gen            NVARCHAR(50),
    dwh_create_date DATETIME DEFAULT GETDATE()
);

IF OBJECT_ID('silver.erp_loc_a101', 'U') IS NOT NULL DROP TABLE silver.erp_loc_a101;
CREATE TABLE silver.erp_loc_a101 (
    cid            NVARCHAR(50),
    cntry          NVARCHAR(50),
    dwh_create_date DATETIME DEFAULT GETDATE()
);

IF OBJECT_ID('silver.erp_px_cat_g1v2', 'U') IS NOT NULL DROP TABLE silver.erp_px_cat_g1v2;
```

```

CREATE TABLE silver.erp_px_cat_g1v2 (
  id          NVARCHAR(50),
  cat         NVARCHAR(50),
  subcat      NVARCHAR(50),
  maintenance NVARCHAR(50),
  dwh_create_date DATETIME DEFAULT GETDATE()
);
GO

```

Step 5.3.2 — Silver Transformation Procedure (proc_load_silver.sql)

The procedure below truncates and repopulates each Silver table, applying the cleansing rules defined in Section 5.2. The customer table example is shown in full; the remaining tables follow the same TRY/CATCH, TRUNCATE + INSERT pattern.

SQL – silver.crm_cust_info

```

-- 1. Loading silver.crm_cust_info
SET @start_time = GETDATE();
PRINT '>> Truncating and Inserting: silver.crm_cust_info';
TRUNCATE TABLE silver.crm_cust_info;

INSERT INTO silver.crm_cust_info (
  cst_id, cst_key, cst_firstname, cst_lastname,
  cst_marital_status, cst_gndr, cst_create_date
)
SELECT
  CAST(cst_id AS INT)          AS cst_id,
  TRIM(cst_key)               AS cst_key,
  TRIM(cst_firstname)         AS cst_firstname,
  TRIM(cst_lastname)         AS cst_lastname,
  CASE
    WHEN UPPER(TRIM(cst_marital_status)) = 'S' THEN 'Single'
    WHEN UPPER(TRIM(cst_marital_status)) = 'M' THEN 'Married'
    ELSE 'n/a'
  END                        AS cst_marital_status,
  CASE
    WHEN UPPER(TRIM(cst_gndr)) = 'F' THEN 'Female'
    WHEN UPPER(TRIM(cst_gndr)) = 'M' THEN 'Male'
    ELSE 'n/a'
  END                        AS cst_gndr,
  TRY_CAST(cst_create_date AS DATE) AS cst_create_date
FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY cst_id ORDER BY cst_create_date DESC) AS flag_last
  FROM bronze.crm_cust_info
  WHERE cst_id IS NOT NULL
) t
WHERE flag_last = 1;

```

SQL – silver.crm_prd_info

```

-- 2. Loading silver.crm_prd_info
TRUNCATE TABLE silver.crm_prd_info;
INSERT INTO silver.crm_prd_info (
  prd_id, cat_id, prd_key, prd_nm, prd_cost, prd_line, prd_start_dt, prd_end_dt
)
SELECT
  CAST(prd_id AS INT)          AS prd_id,
  REPLACE(SUBSTRING(prd_key, 1, 5), '-', '_') AS cat_id,
  SUBSTRING(prd_key, 7, LEN(prd_key))        AS prd_key,
  TRIM(prd_nm)                          AS prd_nm,
  ISNULL(TRY_CAST(prd_cost AS DECIMAL(13,2)), 0) AS prd_cost,

```

```

CASE UPPER(TRIM(prd_line))
  WHEN 'M' THEN 'Mountain'
  WHEN 'R' THEN 'Road'
  WHEN 'S' THEN 'Other Sales'
  WHEN 'T' THEN 'Touring'
  ELSE 'n/a'
END
TRY_CAST(prd_start_dt AS DATE)
TRY_CAST(LEAD(prd_start_dt) OVER (PARTITION BY prd_key ORDER BY prd_start_dt) - 1 AS DATE) AS
prd_end_dt
FROM bronze.crm_prd_info;

```

SQL – silver.crm_sales_details

```

-- 3. Loading silver.crm_sales_details
TRUNCATE TABLE silver.crm_sales_details;
INSERT INTO silver.crm_sales_details (
  sls_ord_num, sls_prd_key, sls_cust_id, sls_order_dt, sls_ship_dt,
  sls_due_dt, sls_sales, sls_quantity, sls_price
)
SELECT
  TRIM(sls_ord_num) AS sls_ord_num,
  TRIM(sls_prd_key) AS sls_prd_key,
  CAST(sls_cust_id AS INT) AS sls_cust_id,
  CASE WHEN LEN(sls_order_dt) = 8 THEN TRY_CAST(sls_order_dt AS DATE) ELSE NULL END AS
sls_order_dt,
  CASE WHEN LEN(sls_ship_dt) = 8 THEN TRY_CAST(sls_ship_dt AS DATE) ELSE NULL END AS
sls_ship_dt,
  CASE WHEN LEN(sls_due_dt) = 8 THEN TRY_CAST(sls_due_dt AS DATE) ELSE NULL END AS
sls_due_dt,
  CASE
    WHEN sls_sales IS NULL OR sls_sales <= 0 OR sls_sales != sls_quantity * ABS(sls_price)
    THEN sls_quantity * ABS(sls_price)
    ELSE sls_sales
  END AS sls_sales,
  CAST(sls_quantity AS INT) AS sls_quantity,
  CASE
    WHEN sls_price IS NULL OR sls_price <= 0
    THEN sls_sales / NULLIF(sls_quantity, 0)
    ELSE sls_price
  END AS sls_price
FROM bronze.crm_sales_details;

```

SQL – silver.erp_cust_az12 / erp_loc_a101 / erp_px_cat_g1v2

```

-- 4. Loading silver.erp_cust_az12
TRUNCATE TABLE silver.erp_cust_az12;
INSERT INTO silver.erp_cust_az12 (cid, bdate, gen)
SELECT
  CASE WHEN cid LIKE 'NAS%' THEN SUBSTRING(cid, 4, LEN(cid)) ELSE cid END AS cid,
  CASE WHEN TRY_CAST(bdate AS DATE) > GETDATE() THEN NULL ELSE TRY_CAST(bdate AS DATE) END AS
bdate,
  CASE
    WHEN UPPER(TRIM(gen)) IN ('F', 'FEMALE') THEN 'Female'
    WHEN UPPER(TRIM(gen)) IN ('M', 'MALE') THEN 'Male'
    ELSE 'n/a'
  END AS gen
FROM bronze.erp_cust_az12;

-- 5. Loading silver.erp_loc_a101
TRUNCATE TABLE silver.erp_loc_a101;
INSERT INTO silver.erp_loc_a101 (cid, cntry)
SELECT

```



```

REPLACE(cid, '-', '') AS cid,
CASE
    WHEN TRIM(cntry) IN ('US', 'USA') THEN 'United States'
    WHEN TRIM(cntry) = 'DE' THEN 'Germany'
    WHEN TRIM(cntry) IS NULL OR TRIM(cntry) = '' THEN 'n/a'
    ELSE TRIM(cntry)
END AS cntry
FROM bronze.erp_loc_a101;

-- 6. Loading silver.erp_px_cat_g1v2
TRUNCATE TABLE silver.erp_px_cat_g1v2;
INSERT INTO silver.erp_px_cat_g1v2 (id, cat, subcat, maintenance)
SELECT TRIM(id), TRIM(cat), TRIM(subcat), TRIM(maintenance)
FROM bronze.erp_px_cat_g1v2;

```

5.4 Execution & Logging

To trigger the entire cleansing and transformation layer:

```

SQL
EXEC silver.load_silver;

```

Sample Log Output

```

Console Output
=====
Loading Silver Layer
=====
>> Truncating and Inserting: silver.crm_cust_info
>> Load Duration: 1 seconds
>> Truncating and Inserting: silver.crm_prd_info
>> Load Duration: 1 seconds
>> Truncating and Inserting: silver.crm_sales_details
>> Load Duration: 2 seconds
>> Truncating and Inserting: silver.erp_cust_az12
>> Load Duration: 1 seconds
>> Truncating and Inserting: silver.erp_loc_a101
>> Load Duration: 1 seconds
>> Truncating and Inserting: silver.erp_px_cat_g1v2
>> Load Duration: 1 seconds
=====
Silver Layer Load Completed Successfully
Total Load Duration: 7 seconds
=====

```

6. Gold Layer — Star Schema

6.1 Overview & Objectives

The Gold Layer serves as the business-ready analytics and reporting zone. It integrates cleansed, standardized data from the Silver Layer into a Star Schema dimensional model.

Unlike the Bronze and Silver layers, which store physical tables, the Gold Layer is constructed using SQL Views. Views present an optimized, decoupled representation of the underlying data without physically duplicating storage, making query execution dynamic and maintenance seamless.

Key Operations Performed

- **Dimensional Modelling:** Organizing entities into primary business subjects — Dimensions (descriptive context) and Facts (quantifiable metrics).
- **Source Integration:** Combining corresponding tables from CRM and ERP sources (e.g., merging silver.crm_cust_info, silver.erp_cust_az12, and silver.erp_loc_a101 into a single Customer Dimension).
- **Surrogate Key Generation:** Assigning unique integer keys (dim_key / fact_key) using functions like ROW_NUMBER() to decouple reporting views from volatile operational source IDs.
- **Bus Architecture Compliance:** Establishing foreign key mapping from facts to dimensions to allow effortless joining in BI tools like Power BI or Tableau.

6.2 Star Schema Diagram



6.3 Dimension & Fact Logic Specifications

6.3.1 gold.dim_customers

Integrates CRM demographic records with ERP birthdate and location attributes.

- **Primary Source:** silver.crm_cust_info
- **Enrichment Sources:** silver.erp_cust_az12 (birthdate, fallback gender) and silver.erp_loc_a101 (country).
- **Key Business Rules:** Generates customer_key via ROW_NUMBER(); resolves gender discrepancies between CRM and ERP, defaulting to CRM when available.

6.3.2 gold.dim_products

Integrates product master data with the product hierarchy categories.

- **Primary Source:** silver.crm_prd_info
- **Enrichment Source:** silver.erp_px_cat_g1v2 (category details via cat_id).
- **Key Business Rules:** Filters out historical/expired products (WHERE prd_end_dt IS NULL) to expose only current product master data; generates product_key via ROW_NUMBER().

6.3.3 gold.fact_sales

Captures granular transactional sales records and links them directly to dimensions via surrogate keys.

- **Primary Source:** silver.crm_sales_details
- **Key Business Rules:** LEFT JOIN to gold.dim_products on sls_prd_key to retrieve product_key; LEFT JOIN to gold.dim_customers on sls_cust_id to retrieve customer_key.

6.4 Implementation Scripts

Step 6.4.1 — Gold Views Creation Script (ddl_gold.sql)

SQL — gold.dim_customers

```
USE DataWarehouse;
GO

IF OBJECT_ID('gold.dim_customers', 'V') IS NOT NULL DROP VIEW gold.dim_customers;
GO
CREATE VIEW gold.dim_customers AS
SELECT
    ROW_NUMBER() OVER (ORDER BY cst.cst_id) AS customer_key, -- Surrogate Key
    cst.cst_id AS customer_id,
    cst.cst_key AS customer_number,
    cst.cst_firstname AS first_name,
    cst.cst_lastname AS last_name,
    loc.cntry AS country,
    cst.cst_marital_status AS marital_status,
    CASE
        WHEN cst.cst_gndr != 'n/a' THEN cst.cst_gndr -- Preferred CRM source
        ELSE ISNULL(ca.gen, 'n/a')
    END AS gender,
    ca.bdate AS birthdate,
    cst.cst_create_date AS create_date
FROM silver.crm_cust_info cst
LEFT JOIN silver.erp_cust_az12 ca ON cst.cst_key = ca.cid
LEFT JOIN silver.erp_loc_a101 loc ON cst.cst_key = loc.cid;
GO
```

SQL — gold.dim_products

```
IF OBJECT_ID('gold.dim_products', 'V') IS NOT NULL DROP VIEW gold.dim_products;
GO
CREATE VIEW gold.dim_products AS
SELECT
    ROW_NUMBER() OVER (ORDER BY prd.prd_start_dt, prd.prd_key) AS product_key, -- Surrogate Key
    prd.prd_id AS product_id,
    prd.prd_key AS product_number,
    prd.prd_nm AS product_name,
    prd.cat_id AS category_id,
    px.cat AS category,
    px.subcat AS subcategory,
    px.maintenance AS maintenance,
    prd.prd_cost AS cost,
    prd.prd_line AS product_line,
    prd.prd_start_dt AS start_date
FROM silver.crm_prd_info prd
LEFT JOIN silver.erp_px_cat_g1v2 px ON prd.cat_id = px.id
WHERE prd.prd_end_dt IS NULL; -- Current product records only
GO
```

SQL — gold.fact_sales

```
IF OBJECT_ID('gold.fact_sales', 'V') IS NOT NULL DROP VIEW gold.fact_sales;
GO
CREATE VIEW gold.fact_sales AS
SELECT
    sd.sls_ord_num AS order_number,
    pr.product_key AS product_key, -- FK to gold.dim_products
    cu.customer_key AS customer_key, -- FK to gold.dim_customers
    sd.sls_order_dt AS order_date,
    sd.sls_ship_dt AS shipping_date,
```

```

        sd.sls_due_dt      AS due_date,
        sd.sls_sales       AS sales_amount,
        sd.sls_quantity   AS quantity,
        sd.sls_price       AS price
FROM silver.crm_sales_details sd
LEFT JOIN gold.dim_products pr ON sd.sls_prd_key = pr.product_number
LEFT JOIN gold.dim_customers cu ON sd.sls_cust_id = cu.customer_id;
GO

```

6.5 Data Quality & Testing Checks

Once the Gold views are established, the following assertion script verifies relationship integrity and checks for missing foreign keys or duplicate surrogate keys (quality_checks_gold.sql). Each check should return zero rows on a healthy warehouse.

```

SQL
USE DataWarehouse;
GO

PRINT '=====';
PRINT 'Executing Gold Layer Quality Verification Checks';
PRINT '=====';

-- 1. Uniqueness Test: duplicate surrogate keys in Dimensions
PRINT '>> Check 1: Verifying Surrogate Key Uniqueness in gold.dim_customers';
SELECT customer_key, COUNT(*) AS duplicate_count
FROM gold.dim_customers
GROUP BY customer_key
HAVING COUNT(*) > 1;

PRINT '>> Check 2: Verifying Surrogate Key Uniqueness in gold.dim_products';
SELECT product_key, COUNT(*) AS duplicate_count
FROM gold.dim_products
GROUP BY product_key
HAVING COUNT(*) > 1;

-- 2. Foreign Key Referential Integrity Test in Fact Table
PRINT '>> Check 3: Identifying Unmapped Products in gold.fact_sales';
SELECT COUNT(*) AS unmapped_product_records
FROM gold.fact_sales
WHERE product_key IS NULL;

PRINT '>> Check 4: Identifying Unmapped Customers in gold.fact_sales';
SELECT COUNT(*) AS unmapped_customer_records
FROM gold.fact_sales
WHERE customer_key IS NULL;

PRINT '=====';
PRINT 'Quality Checks Completed';
PRINT '=====';

```

7. Summary

This project delivered a complete, end-to-end Data Warehouse built on SQL Server using the Medallion Architecture. Raw CSV data from two independent source systems — CRM and ERP — was ingested into a Bronze staging layer, cleansed and standardized in a Silver layer, and finally modeled into a business-ready Star Schema in the Gold layer.

Outcomes

- A fully automated, idempotent ETL pipeline driven by stored procedures with built-in error handling and execution logging.
- A cleansed, standardized dataset with consistent types, unified categorical codes, and resolved cross-system keys.
- A Star Schema (gold.dim_customers, gold.dim_products, gold.fact_sales) optimized for fast, simple BI reporting and analysis.
- Automated data quality checks validating surrogate key uniqueness and referential integrity end-to-end.

Skills Demonstrated

- Data Architecture: layer separation, schema design, and Medallion Architecture principles.
- Data Engineering: T-SQL ETL scripting, data cleansing, and robust error handling.
- Data Modelling: dimensional modelling, surrogate keys, and Star Schema design for analytics.